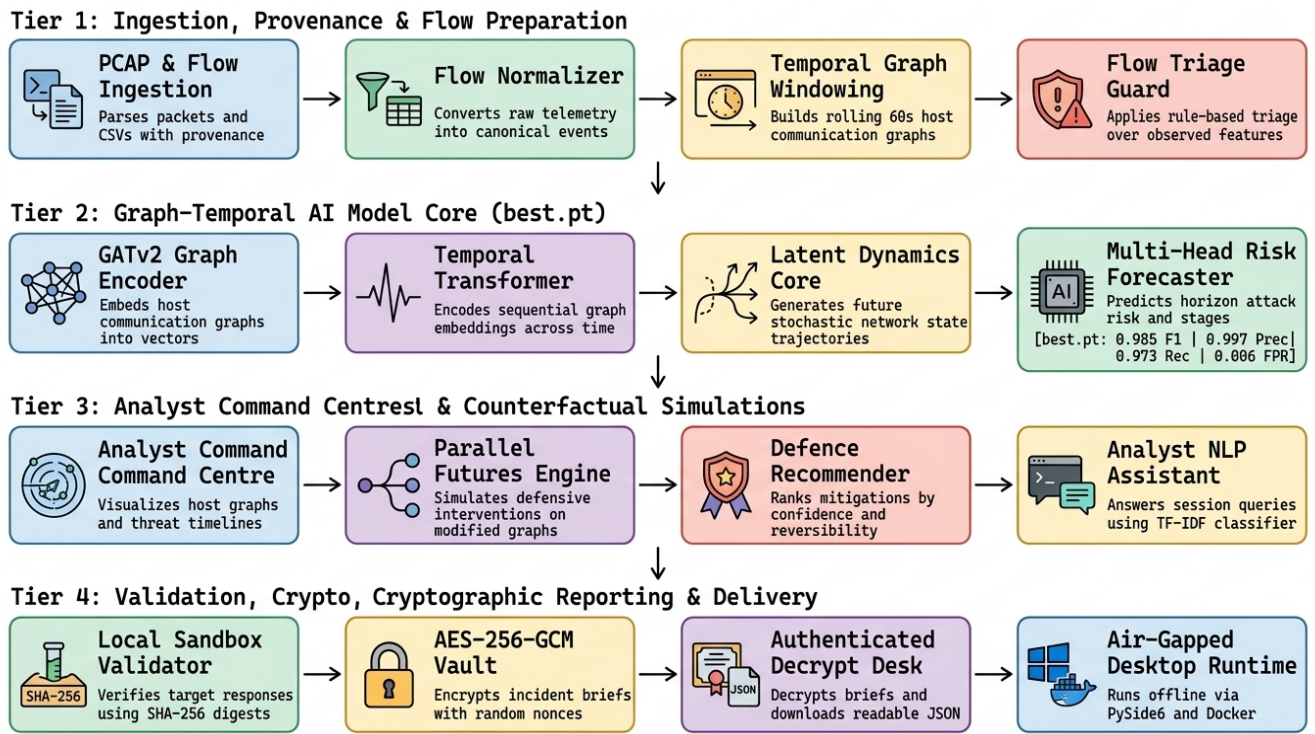


CYBERMIND Architecture Document

Smart India Hackathon 2026 | Problem Statement SIH26153 | CYBERMIND

System Overview

CYBERMIND uses a four-tier local architecture that separates telemetry preparation, learned forecasting, analyst decision support, and trusted delivery. All core processing runs locally. The diagram below is the authoritative visual architecture and shows the main left-to-right data path.



Architecture Layers

Tier	Components	Responsibility
1 Ingestion	PCAP and flow ingestion, normaliser, temporal graph windowing, flow triage guard	Parses local captures or CSV files, preserves provenance, creates canonical flow events and rolling host-communication graphs, and keeps observed-flow triage separate from AI forecasts.
2 Model Core	GATv2 encoder, temporal Transformer, latent dynamics, risk and stage heads	Encodes recent graph history and generates stochastic future trajectories, future graph-window risk, coarse attack-stage estimates, uncertainty cues, and model identity.
3 Analyst Layer	Command Centre, Parallel Futures, defence recommender, offline NLP assistant	Links evidence to forecasts, compares in-memory defensive alternatives, ranks reversible suggestions, and answers bounded questions about the loaded case.
4 Trust and Delivery	Local sandbox validator, AES-256-GCM vault, decrypt desk, desktop and Docker runtime	Records bounded response evidence, protects incident briefs with authenticated encryption, supports in-app decryption, and keeps core processing local.

End to End Flow

PCAP or CSV -> canonical flows -> temporal graph windows -> best.pt forecast -> analyst simulation and validation -> encrypted incident brief -> authenticated in-app decryption

Operational boundaries. The trained model forecasts aggregate graph windows, not verified malicious labels for individual flows. Parallel Futures compares model outputs on modified graph copies and does not prove causal impact. The local validator records target responses but does not retrain the model. Consequential defensive actions remain under analyst control.

Detailed Component Architecture

Smart India Hackathon 2026 | SIH26153 | CYBERMIND Technical Detail

1 Ingestion and State Construction

The analyst manually selects a `.pcap`, `.pcapng`, `.cap`, or `.csv` file. The FastAPI ingestion service records source provenance, extracts packets or matches flow columns, and converts the input into a canonical event schema containing endpoints, ports, protocol, timestamps, packet and byte counts, duration, and optional source labels. Missing CSV timestamps are handled only through a disclosed assumed spacing; labels are never invented for raw captures.

Canonical events are grouped into overlapping 60-second windows with a 30-second stride. Each window becomes a directed host-communication graph: hosts are nodes, communications are edges, and numeric traffic statistics form node and edge features. At least three graph windows are required for forecasting. The flow-triage guard separately derives Critical, High, Medium, secured, or Unassessed investigation hints from observed features and available source labels; these indicators are not predictions from `best.pt`.

2 Graph Temporal Forecasting Engine

Stage	Processing	Output
Graph encoder	GATv2 applies attention over each host-communication graph and optionally consumes edge features stored in the checkpoint configuration.	One learned embedding per observed graph window.
Temporal encoder	A Transformer reads the ordered graph embeddings and represents the recent evolution of the network.	Latent observed state z_t .
Dynamics rollout	A learned diagonal-Gaussian transition model samples multiple future latent trajectories from the current state.	Multi-step stochastic future states and dispersion.
Prediction heads	Risk, coarse-stage, and future-state heads decode every forecast horizon. Saved training normalization is applied before inference.	Current and horizon risk, stage distribution, confidence and OOD proxies.

3 Analyst and Counterfactual Services

The React Command Centre retrieves events, graph snapshots, forecasts, cases, and validation records through local REST endpoints and WebSocket updates. Threat Forecast presents the multi-step trajectory with model identity and uncertainty proxies. Attack Graph reconnects forecast context to observed hosts and edges. The offline NLP assistant uses a bounded TF-IDF and intent-classification pipeline to answer read-only questions from the loaded session; it cannot scan a network or modify evidence.

Parallel Futures copies the current graph, applies an allowed in-memory mutation such as host isolation, port blocking, or rate limiting, and reruns the same forecasting model. The defence recommender compares each simulated trajectory with the no-action baseline using predicted risk change, confidence, reversibility, and collateral-impact heuristics. Results are model-based comparisons for analyst review, not measured causal effects or automatically executed firewall rules.

4 Validation Security and Deployment

Subsystem	Implementation and Trust Boundary
Local validation	Freezes a forecast, calls four bounded endpoints on a bundled sandbox target, and stores HTTP status codes plus SHA-256 response-body hashes. It verifies target responses but does not establish attack-stage truth or retrain the model.
Protected reports	Web Crypto generates a fresh random 256-bit key and 96-bit nonce. AES-256-GCM encrypts the incident brief locally. The JSON envelope contains nonce and ciphertext, not the key; the recipient imports it and supplies the separately shared key inside CYBERMIND.
Windows runtime	CYBERMIND.exe packages the React assets, FastAPI backend, dependencies, checkpoint, and PySide6/Qt WebEngine window. The service binds to an ephemeral loopback address and requires no cloud model endpoint.
Docker runtime	Prebuilt Linux x86-64 images run through Docker Compose, bind the application to 127.0.0.1:8000, and retain records in a named volume. Bundled launch scripts load images locally without pulling at runtime.

Interface Contracts

Input: local packet capture or flow CSV. **Intermediate state:** canonical events, graph windows, model metadata, case records, and frozen validation evidence. **Output:** graph-window risk trajectory, coarse stage distribution, uncertainty proxies, counterfactual comparisons, and an optional encrypted incident brief. A missing or invalid checkpoint makes forecasting unavailable rather than generating substitute scores.